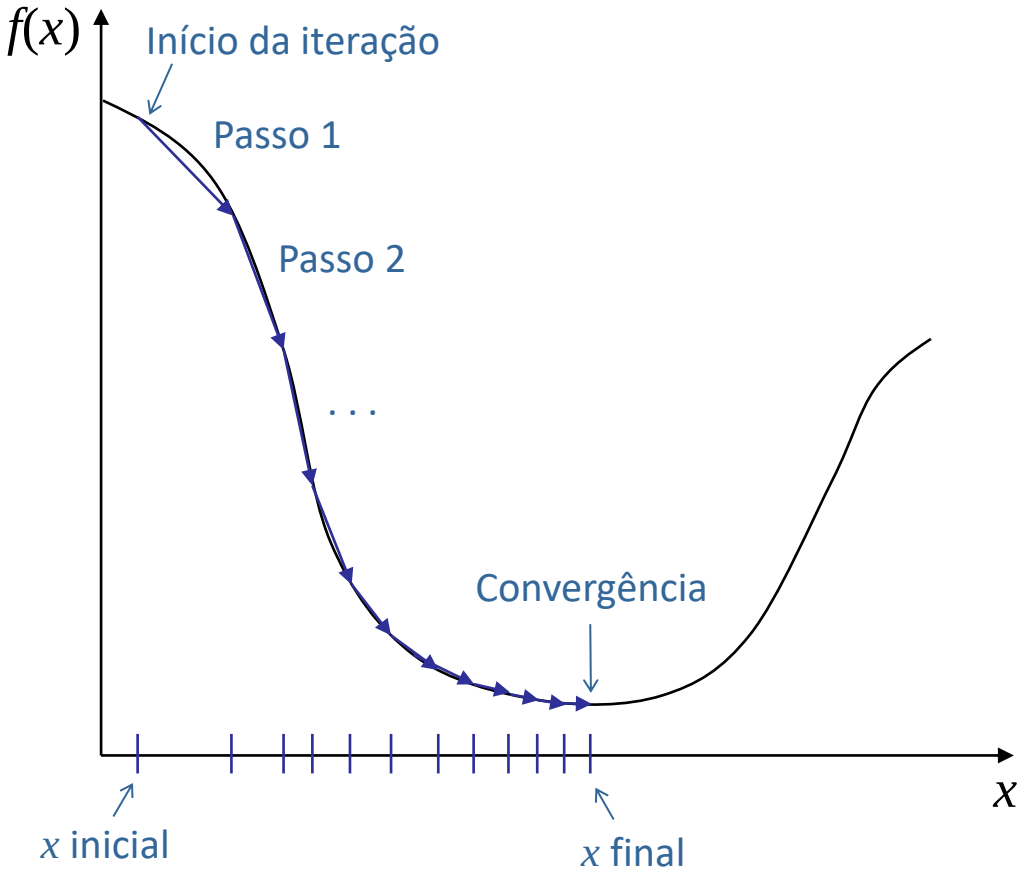
REDES NEURONAIS ARTIFICIAIS

Luís Morgado

ISEL-DEI

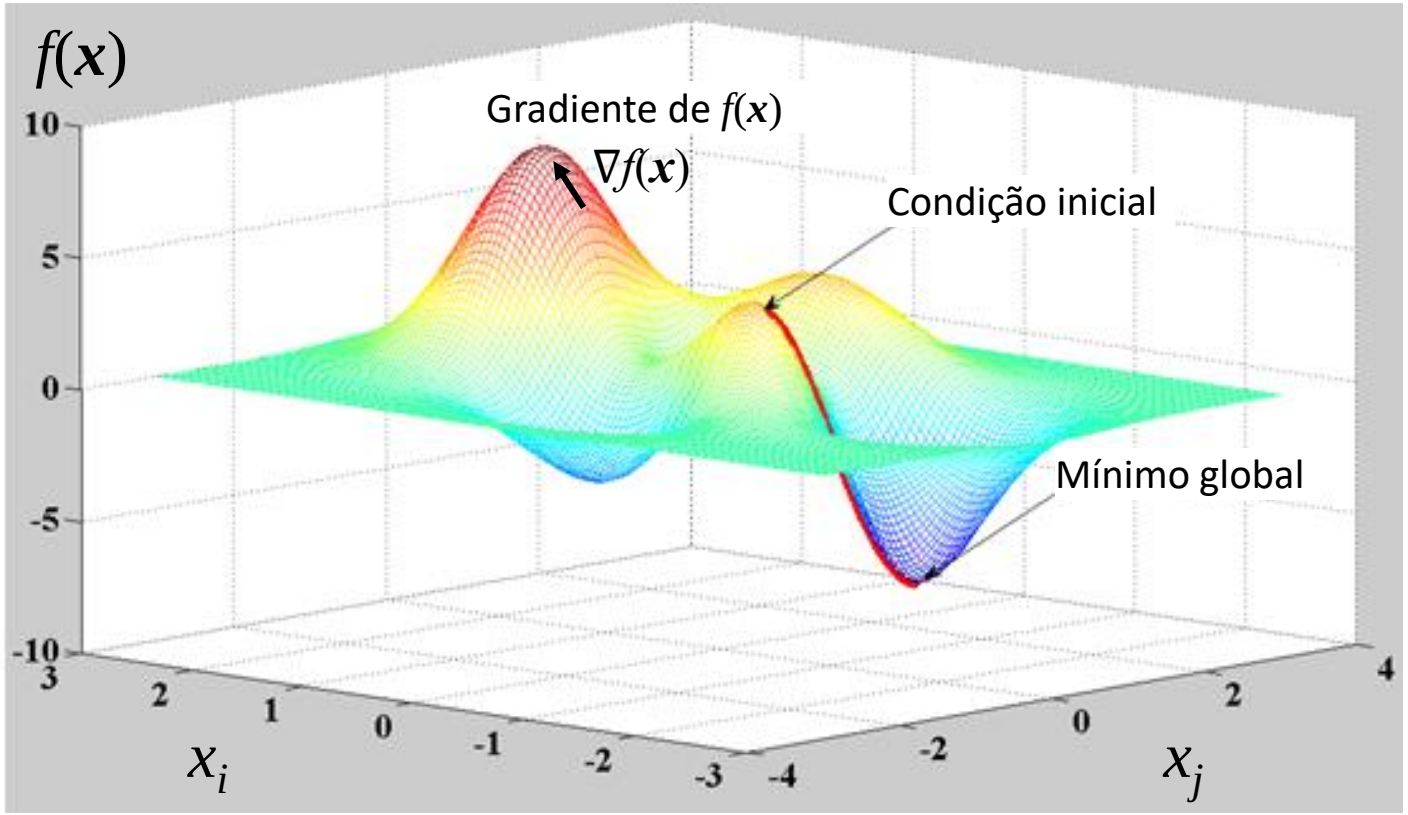
ALGORITMO DE DESCIDA DE GRADIENTE

O valor de um parâmetro x de uma função $f(x)$ é actualizado iterativamente com base no gradiente da função, no sentido oposto ao do gradiente (minimização), até ser atingido um mínimo dessa função (convergência)



ALGORITMO DE DESCIDA DE GRADIENTE

Objectivo: Minimizar $f(x)$



$$\boldsymbol{x}^{n+1} = \boldsymbol{x}^{n} + \Delta\boldsymbol{x}^{n}$$

$$\Delta\boldsymbol{x}^{n} = -\alpha\nabla f(\boldsymbol{x})$$

$n \geq 0$ – Índice do passo

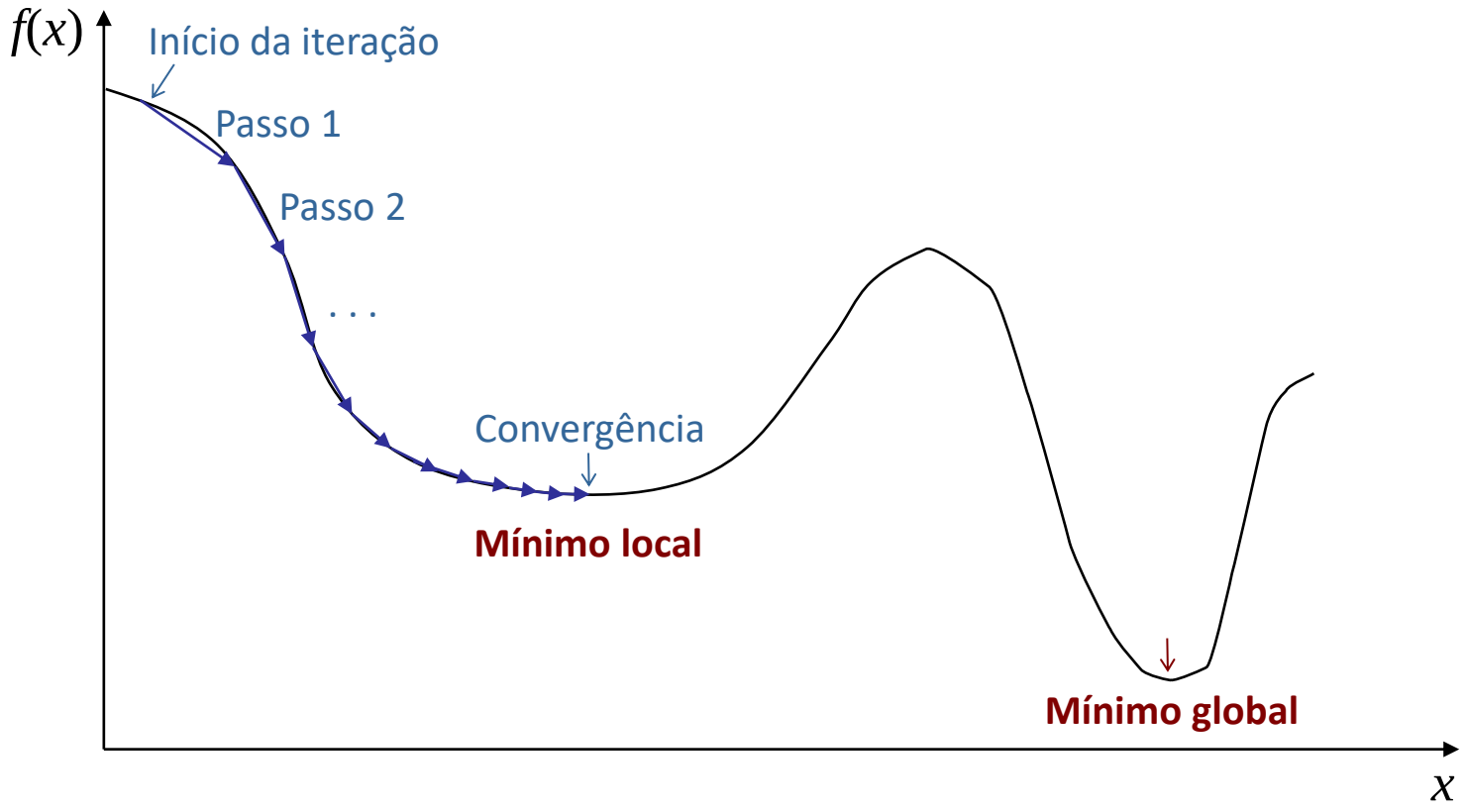
α – Dimensão do passo

$\Delta\boldsymbol{x}$ – Variação de $\boldsymbol{x}$ para actualização

ALGORITMO DE DESCIDA DE GRADIENTE

Problema dos óptimos locais

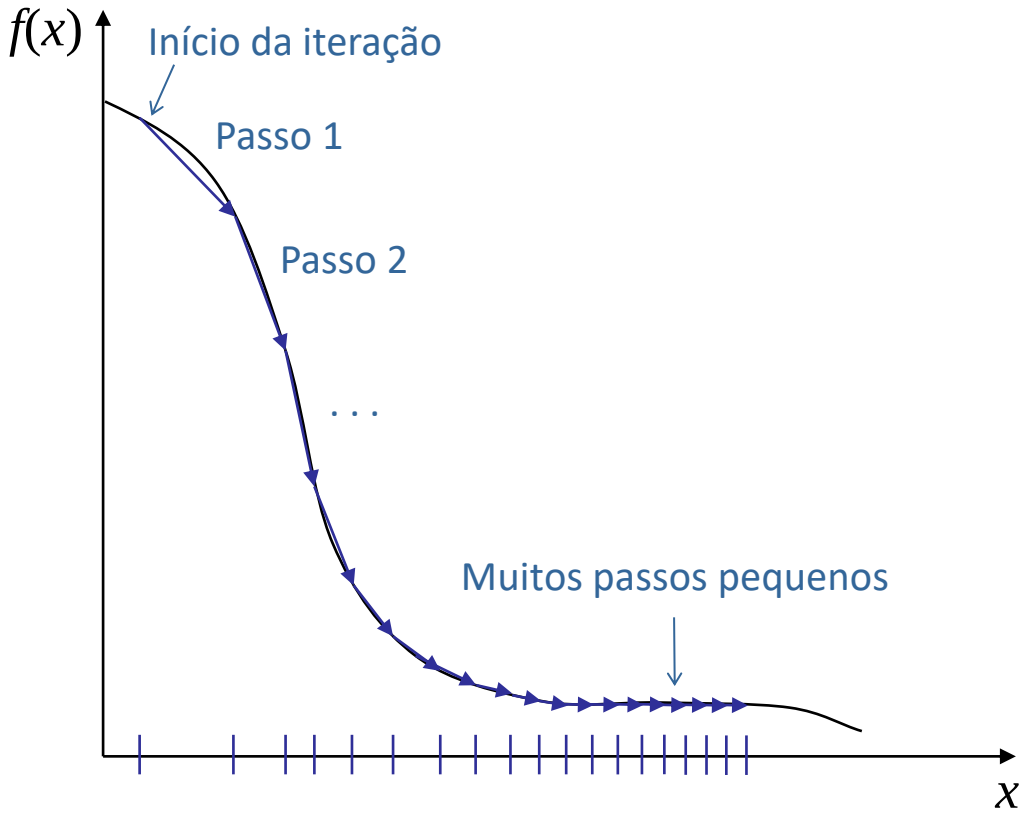
O algoritmo de descida de gradiente não dá garantias acerca do mínimo encontrado ser global ou local



ALGORITMO DE DESCIDA DE GRADIENTE

Problema da convergência lenta

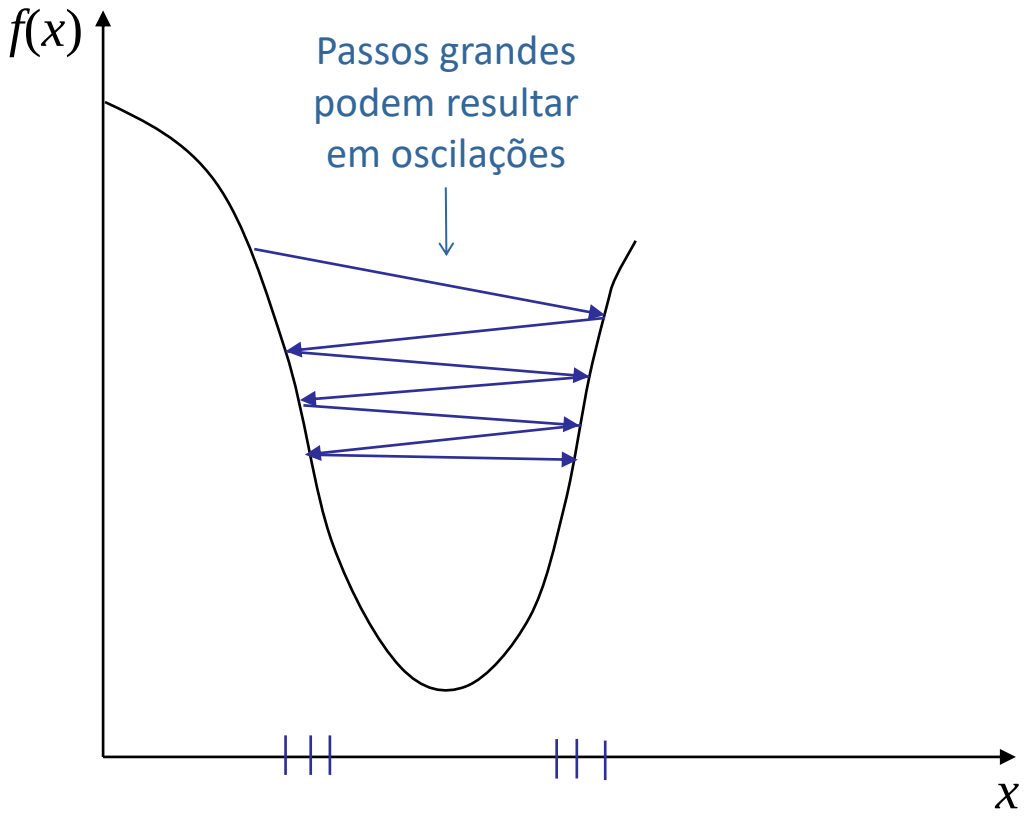
O algoritmo de descida de gradiente pode ter uma convergência lenta em zonas com gradiente pequeno ou nulo ou se o passo α for pequeno



ALGORITMO DE DESCIDA DE GRADIENTE

Problema da oscilação

O algoritmo de descida de gradiente pode ter uma convergência lenta ou mesmo ser impossível a convergência se o passo α for demasiado grande

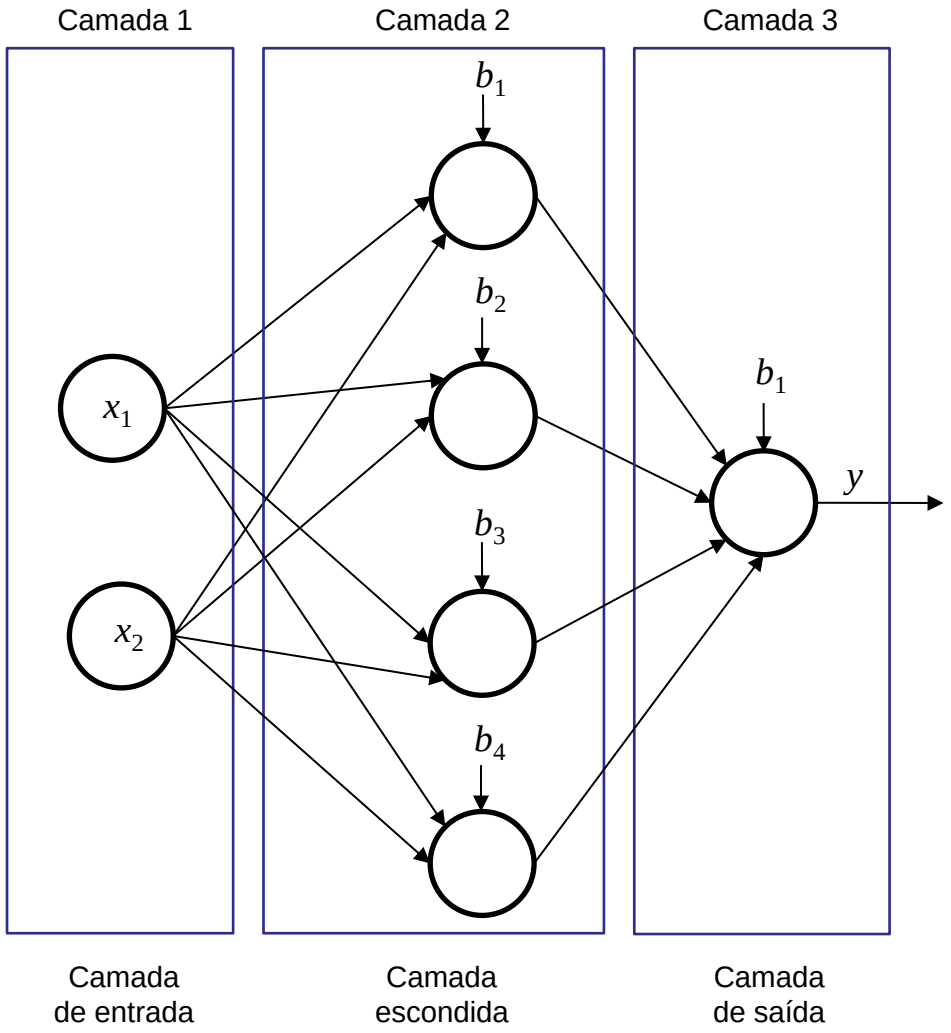


EXEMPLO: APRENDIZAGEM DA FUNÇÃO XOR

Função XOR

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

Forma da rede = [2, 4, 1]



EXEMPLO: APRENDIZAGEM DA FUNÇÃO XOR

Dados de treino e de teste

entradas = [[0,0], [0,1], [1,0], [1,1]]

saídas = [[0], [1], [1], [0]]

Parâmetros de estudo

Forma = [2, 4, 1]

Taxa de aprendizagem = 0.1

Erro máximo = 0.05

Função de activação = tanh

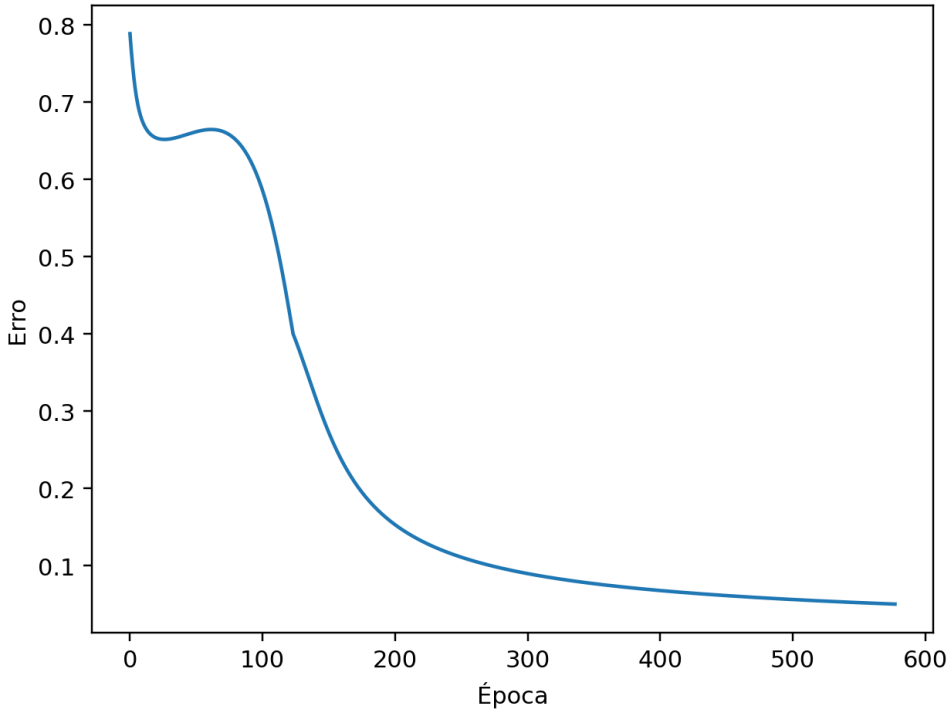
Erro final

Época: 577, *Erro*: 0.04998

Previsão da rede

[[0.00223864944024421 1],
[0.9501690172501429],
[0.9521095001628983],
[0.0018339883898856608]]

Evolução do erro de classificação
ao longo do treino da rede



ALGORITMO DE RETROPROPAGAÇÃO

Exemplo: Problema da oscilação

Convergência lenta ou impossibilidade de convergência devido ao passo α (taxa de aprendizagem) ser demasiado grande

Dados de treino e de teste

entradas = [[0,0], [0,1], [1,0], [1,1]]

saídas = [[0], [1], [1], [0]]

Parâmetros de estudo

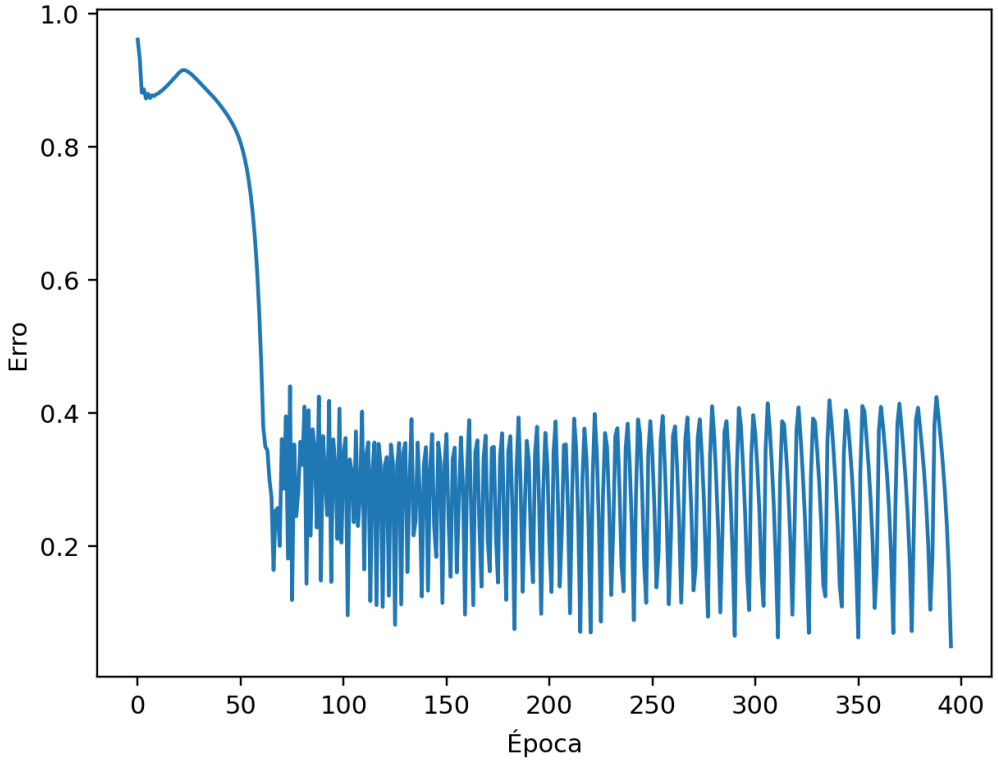
Forma = [2, 4, 1]

Taxa de aprendizagem = 0.5

Erro máximo = 0.05

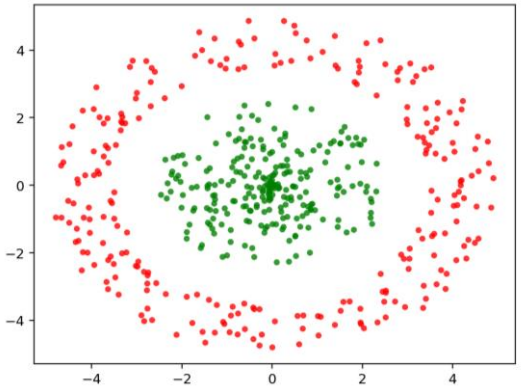
Função de activação = tanh

Evolução do erro de classificação ao longo do treino da rede



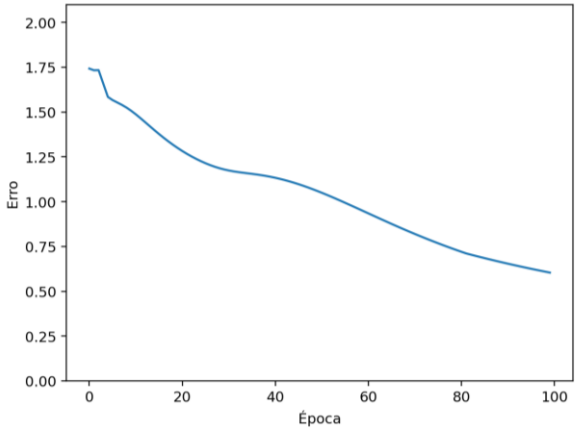
EXEMPLO: PROBLEMA RADIAL

Resultados para diferentes valores da taxa de aprendizagem α

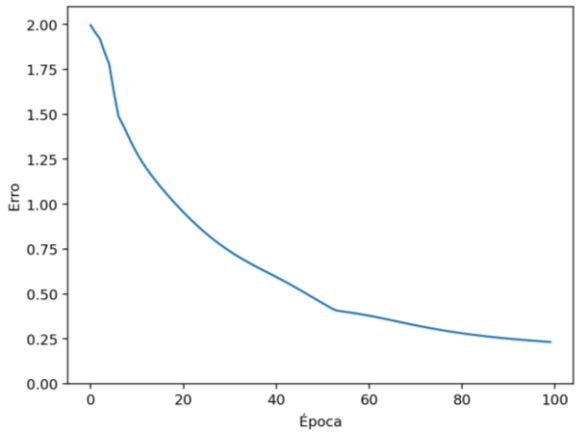


Problema radial
com 500 amostras

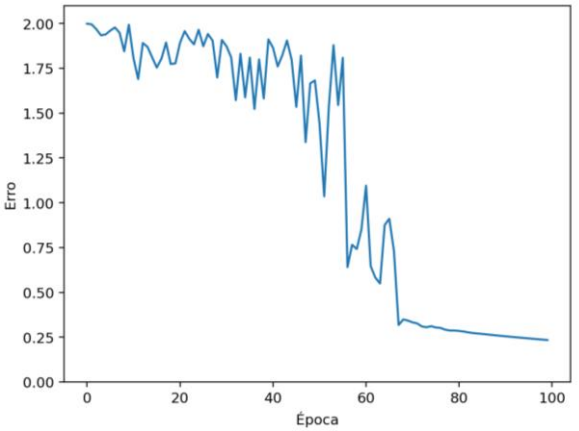
α = 0.01



α = 0.02



α = 0.1



ALGORITMO DE DESCIDA DE GRADIENTE

Descida de gradiente com momento

Para acelerar a convergência do algoritmo de descida de gradiente, é possível introduzir um termo de momento de modo a reduzir as amplitudes de variação dos parâmetros da rede, o que possibilita ultrapassar regiões menos favoráveis do espaço de pesos, como regiões planas ou de baixo gradiente e mínimos locais

- O momento opera como uma forma de memória de variação dos valores dos parâmetros, possibilitando ter em conta os valores de actualizações anteriores desses parâmeros
- Para esse efeito é considerado um termo de momento que expressa a influência dos valores das actualizações anteriores

$$\boldsymbol{x}^{n+1} = \boldsymbol{x}^{n} + \Delta\boldsymbol{x}^{n}$$

$$\Delta\boldsymbol{x}^{n} = -\alpha\nabla f(\boldsymbol{x}) + \underbrace{\beta\Delta\boldsymbol{x}^{n-1}}_{\text{Termo de momento}}$$

Expressa uma memória da variação anterior do parâmetro $\boldsymbol{x}$

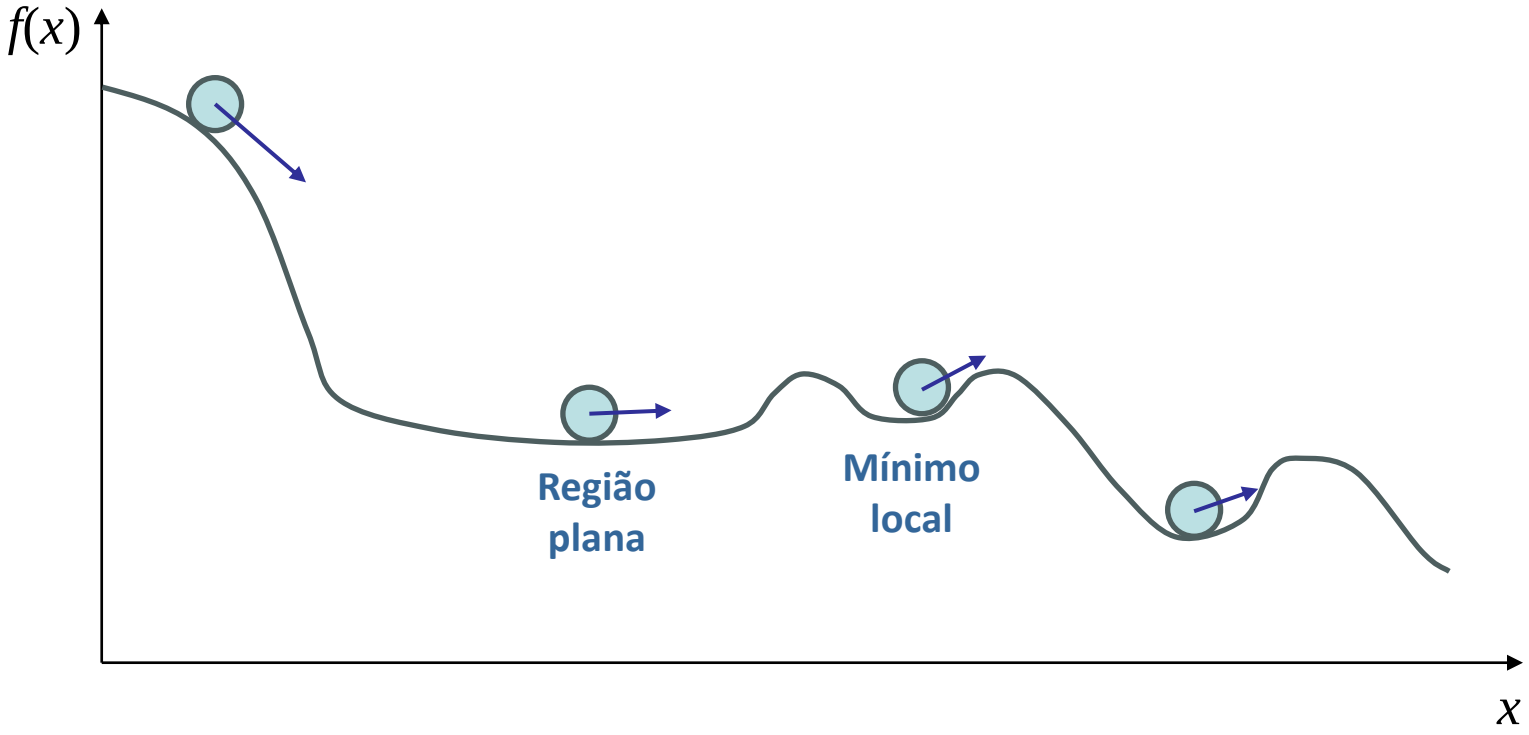
$n \geq 0$ – Índice do passo
α – Dimensão do passo
β – Factor de momento
$\Delta\boldsymbol{x}$ – Variação de $\boldsymbol{x}$ para actualização

ALGORITMO DE DESCIDA DE GRADIENTE

Descida de gradiente com momento

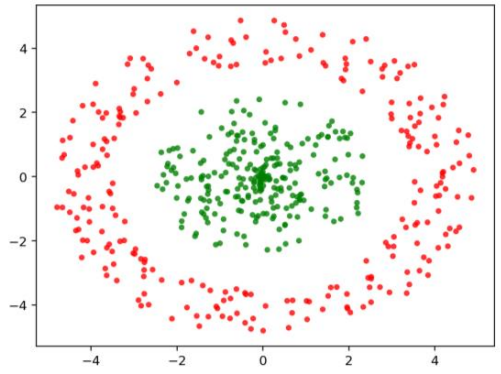
O momento traduz-se numa inércia de deslocação que possibilita ultrapassar regiões planas, de gradiente baixo ou nulo, bem como óptimos locais, facilitando a convergência do processo de optimização dos parâmetros de uma rede neuronal

No entanto, não resolve o problema dos óptimos locais



DESCIDA DE GRADIENTE COM MOMENTO

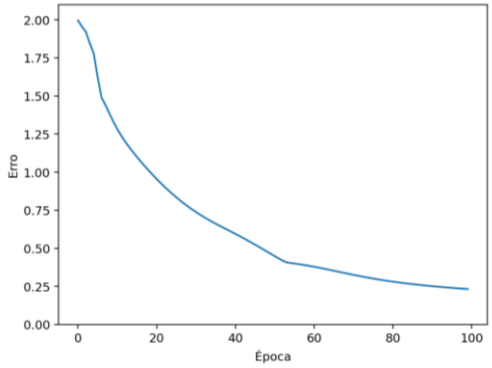
Exemplo:



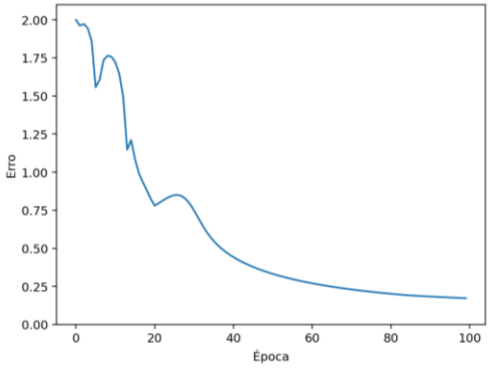
Problema radial
com 500 amostras

Resultados para diferentes valores do factor de momento β com $\alpha = 0.02$

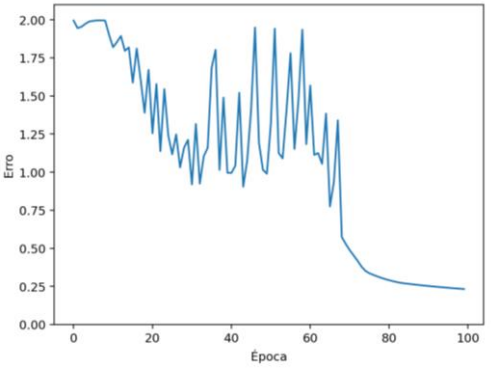
$\beta = 0.0$



$\beta = 1.0$



$\beta = 2.0$



DESCIDA DE GRADIENTE ESTOCÁSTICA

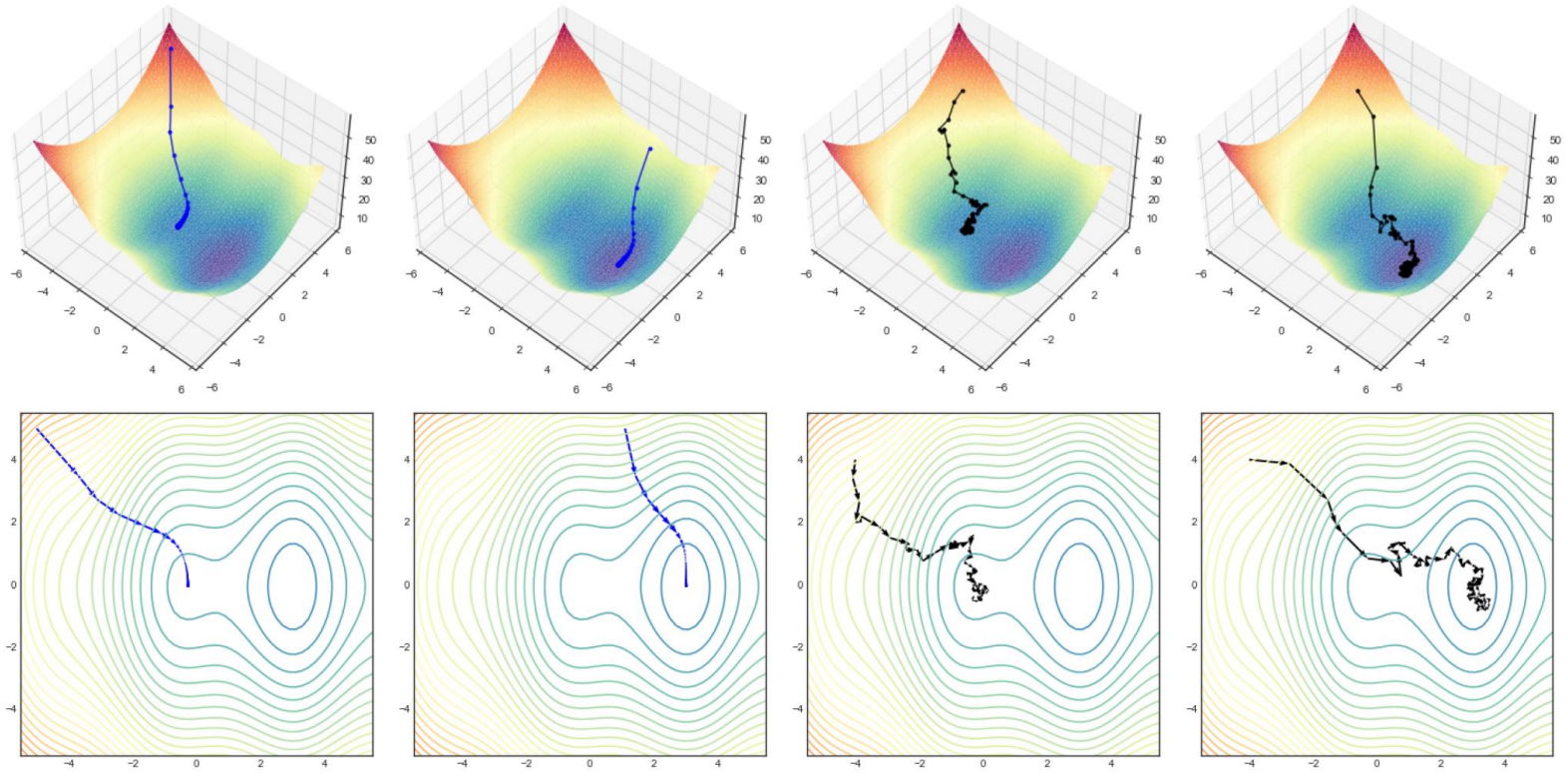
A *descida de gradiente estocástica* (*Stochastic Gradient Descent* - SGD) é uma variante do algoritmo de descida de gradiente, no qual **é utilizada aleatoriedade para ultrapassar mínimos locais**

- Em cada época de treino é utilizado apenas um subconjunto aleatório (*batch*) do conjunto de dados
- As actualizações dos parâmetros são feitas após cada subconjunto (*batch*), resultando num processo de treino mais rápido e que introduz uma componente aleatória na exploração do espaço de pesos

DESCIDA DE GRADIENTE ESTOCÁSTICA

Exemplo: Desempenho dos algoritmos de descida de gradiente (DG) e de descida de gradiente estocástica (DGE), para uma função objectivo com um mínimo local próximo do mínimo global

Consoante a condição inicial (DG) e das flutuações resultantes da componente estocástica (DGE), pode ser possível ou não atingir o mínimo global



Descida de gradiente (DG) | Descida de gradiente estocástica (DGE)

Adaptado de [Berner *et al.*, 2023]

DESCIDA DE GRADIENTE ESTOCÁSTICA

Vantagens:

- Mais eficiente em termos de uso de recursos, pois não precisa processar todo o conjunto de dados a cada iteração
- O processo de treino pode convergir mais rapidamente devido a actualizações mais frequentes (menos repetições das amostras)
- Mais adequada para grandes conjuntos de dados, como em tarefas de classificação de imagens ou de processamento linguístico

Desvantagens:

- A trajectória de convergência pode ser mais irregular, com oscilações na função de custo
- A aleatoriedade pode ajudar a ultrapassar mínimos locais, mas também pode dificultar a convergência em direcção ao mínimo global

APRENDIZAGEM EM REDES NEURONAIS

Métodos de Optimização para Treino de Redes Neuronais

Descida de Gradiente

$$\Delta w_i = -\alpha \frac{\partial L}{\partial w_i}$$

Descida de Gradiente com Momento

$$\Delta w_i^{t+1} = -\alpha \frac{\partial L}{\partial w_i} + \beta \Delta w_i^t$$

Descida de Gradiente Estocástica (SGD - *Stochastic Gradient Descent*)

Em cada iteração (época) de treino é utilizado um subconjunto aleatório do conjunto de treino (*batch*)

Outros métodos

Root Mean Square Propagation (RMSProp), *Adaptive Moment Estimation* (Adam), *Adaptive Gradient Algorithm* (AdaGrad)

PROCESSO DE APRENDIZAGEM

O processo de aprendizagem com base em redes neuronais envolve o pré-processamento dos dados para garantir a sua adequação e qualidade, a extracção de características relevantes para a aprendizagem e o treino das redes por meio da optimização dos seus parâmetros, com as seguintes etapas principais:

- **Pré-processamento dos dados**
  - Preparação dos dados fonte com codificação de variáveis para serem utilizadas na rede neuronal
- **Extracção de características**
  - Selecção de características com base nas variáveis já codificadas e eventual definição de novas variáveis derivadas
- **Treino da rede neuronal**
  - Treino da rede neuronal com base nos dados preparados e seleccionados
- **Avaliação e adequação da rede**
  - Após o treino, os resultados obtidos são avaliados de modo adequar a arquitectura da rede, os hiper-parâmetros definidos ou a realizar mais pré-processamento e extracção de características, se necessário

PROCESSO DE APRENDIZAGEM

Pré-processamento dos dados

- Etapa que consiste na preparação dos dados fonte para serem utilizados na rede neuronal, nomeadamente:
  - **Limpeza de dados**
    - Remoção de dados inconsistentes, redundantes ou irrelevantes, o que pode incluir a eliminação de amostras ou a correcção de erros nos dados
  - **Normalização dos dados**
    - Adaptação dos dados para que tenham dimensões de valores consistentes
  - **Codificação dos dados**
    - Conversão dos formatos dos dados fonte para representações numéricas adequadas à arquitectura das redes utilizadas
  - **Separação dos dados**
    - Separação dos dados em conjuntos específicos, nomeadamente de treino e de teste (para validação e ajuste de hiper-parâmetros)

PROCESSO DE APRENDIZAGEM

Extracção de características

- Etapa que consiste na selecção de características com base nas variáveis já codificadas e eventual definição de novas características derivadas:
  - **Selecção de características**
    - Identificação de quais as características que são mais relevantes para a tarefa a aprender
  - **Definição de características derivadas**
    - Definição de novas características a partir das existentes que possam ajudar a melhorar o desempenho da rede, nomeadamente:
      - Combinação de variáveis
      - Transformação de variáveis
        - Por exemplo, transformações matemáticas, como a introdução de não linearidades para facilitar a aprendizagem de relações não lineares entre as características e as variáveis alvo

PROCESSO DE APRENDIZAGEM

Treino da rede

- Etapa de aprendizagem da rede com base nos dados preparados e seleccionados, tendo por base os seguintes aspectos principais:
  - **Arquitectura da rede**
    - Definição da estrutura (forma) da rede neuronal, nomeadamente, o número de camadas e a organização de cada camada (dimensão, função de activação dos neurónios, conectividade)
  - **Função de perda**
    - Definição da função de perda (*erro*) adequada para a tarefa a aprender
  - **Método de optimização**
    - Definição do método de optimização adequado para a tarefa a aprender
  - **Número de épocas de treino e dimensão dos conjuntos de treino**
    - Definição do número de épocas (iterações sobre o conjunto de treino) e o tamanho dos subconjuntos (*batch*) de treino (número de amostras do conjunto de treino processadas para cada actualização dos pesos)

ARQUITECTURA DE UMA REDE NEURONAL

A definição da arquitectura de uma rede neuronal para a aprendizagem de uma determinada tarefa é uma actividade que inclui uma componente experimental, pois normalmente não são completamente conhecidas as características das representações a aprender a partir dos dados

Alguns aspectos principais da definição da arquitectura de uma rede neuronal são:

- Número de neurónios de entrada
- Número de neurónios de saída
- Número de neurónios escondidos
- Número de camadas escondidas

ARQUITECTURA DE UMA REDE NEURONAL

- **Número de neurónios de entrada**
  - O número de neurónios de entrada deve corresponder ao número de características definidas como relevantes para a tarefa a aprender
- **Número de neurónios de saída**
  - Depende do tipo de tarefa a aprender, nomeadamente:
    - **Classificação Binária**
      - Existe um único neurónio de saída
    - **Classificação multi-classe**
      - O número de neurónios de saída deve ser igual ao número de classes a discriminar
    - **Regressão**
      - Existe um único neurónio de saída que representa o valor a prever

ARQUITECTURA DE UMA REDE NEURONAL

- **Número de neurónios escondidos**
  - Normalmente não são completamente conhecidas as características das representações a aprender a partir dos dados, pelo que não existe uma forma precisa de definir o número de neurónios escondidos de uma rede
  - Por esse motivo, a vertente experimental é importante, por exemplo, definindo uma arquitectura inicial com um número menor de neurónios escondidos e aumentar gradualmente esse número
  - Heurísticas comuns para o número inicial de neurónios escondidos:
    - Valor intermédio entre o número de entradas e o número de saídas

      número de neurónios escondidos ≈ (*número de entradas* + *número de saídas*) / 2
    - Para problemas de grande complexidade

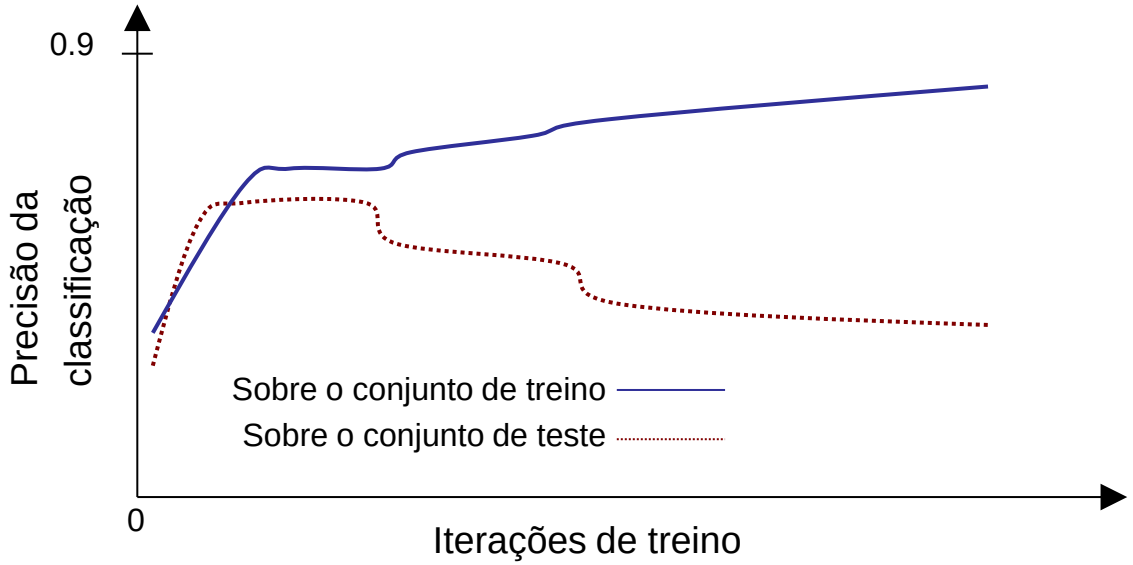
      número de neurónios escondidos ≥ *número de entradas*
  - O desempenho das diferentes configurações da rede deve ser progressivamente verificado até serem atingidos resultados adequados

ARQUITECTURA DE UMA REDE NEURONAL

- **Número de camadas escondidas**
  - Uma rede neuronal com uma única camada escondida, com um número finito de neurónios, pode aproximar qualquer função contínua num intervalo compacto
  - A existência de múltiplas camadas pode tornar o processo de aprendizagem mais eficiente
  - Redes mais profundas (com mais camadas) têm a capacidade de aprender representações mais complexas,
  - No entanto, redes com mais camadas são também mais propensas à ocorrência de *sobreparametrização* (adaptação excessiva aos dados de treino, limitando a capacidade de generalização)

O PROBLEMA DA SOBREPARAMETRIZAÇÃO

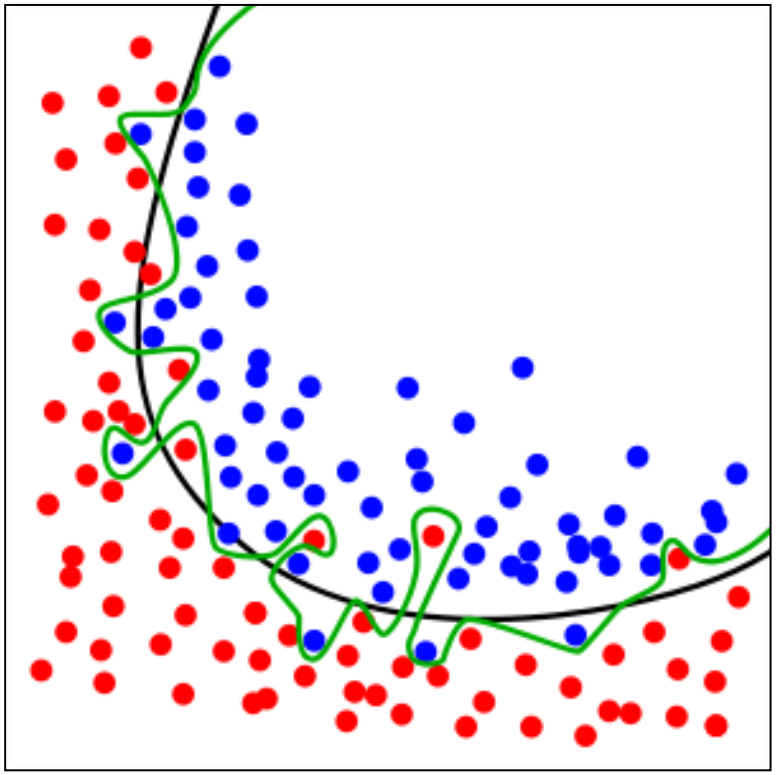
- A **sobreparametrização**, ou sobreadequação **(*overfitting*)** é um dos problemas que pode surgir com frequência em redes neuronais artificiais
- Por exemplo, entre outras causas, se a aprendizagem se prolongar em demasia, pode resultar numa **diminuição da capacidade de generalização da rede**
- Apesar da rede ter aprendido os detalhes de um conjunto de treino particular, pode não responder de forma adequada quando são apresentados exemplos novos



SOBREPARAMETRIZAÇÃO

Uma rede pode aprender os detalhes de um conjunto de treino particular, não respondendo de forma adequada quando são apresentados exemplos novos

Exemplo de amostras de treino
representadas num espaço bidimensional



Exemplos de fronteiras de classificação para duas redes distintas

Rede sem sobreparametrização

Rede com sobreparametrização

SOBREPARAMETRIZAÇÃO

- **Causas**
  - Aprendizagem prolongada em demasia
  - Poucos exemplos de treino
  - Dados com ruído
  - Rede tem demasiados parâmetros (neurónios, ligações)
    - Mais do que necessário para os dados a processar

SOBREPARAMETRIZAÇÃO

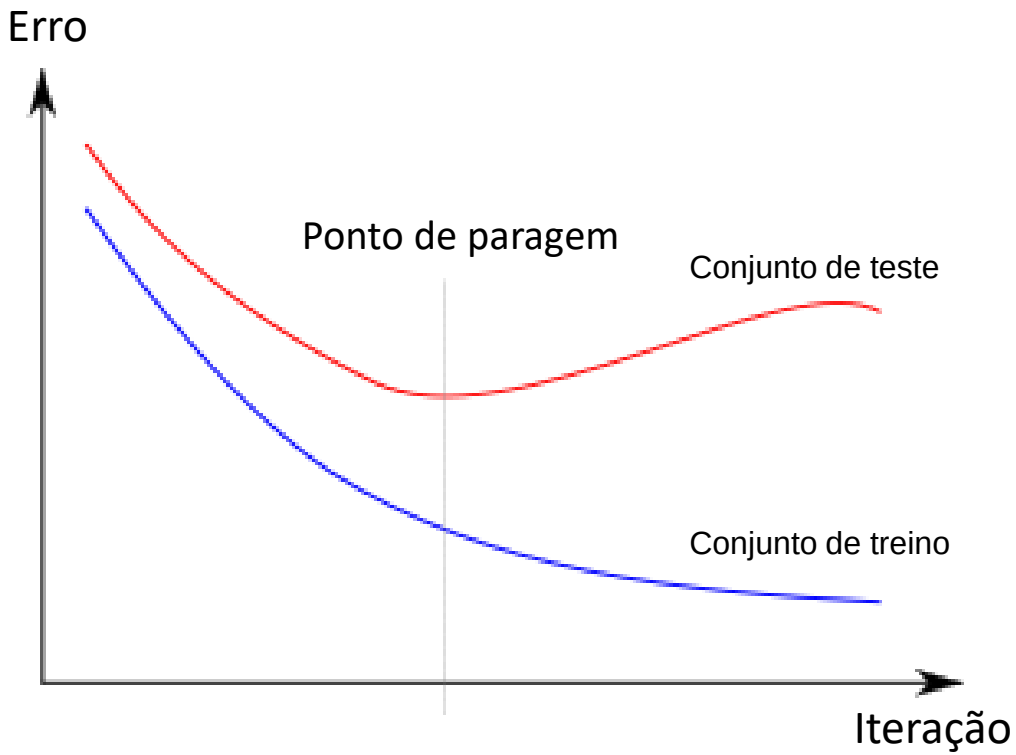
- **Soluções**
  - Simplificação da rede
  - Paragem prévia (*early stopping*)
  - Aumento dos dados (*data augmentation*)
  - Regularização
  - Redução selectiva de parâmetros (*dropout*)

SOBREPARAMETRIZAÇÃO

Paragem prévia

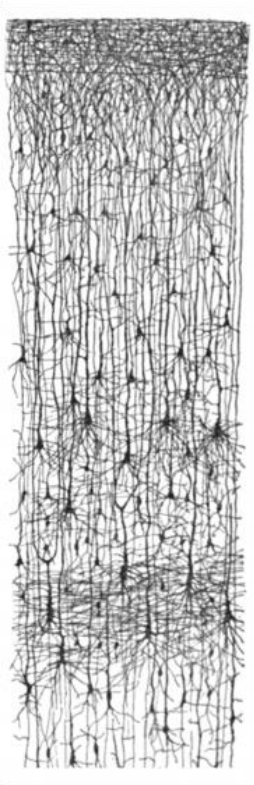
São utilizados dois conjuntos de dados, um *conjunto de treino* e um *conjunto de teste*

A aprendizagem é finalizada quando ocorre a degradação do desempenho da rede relativo ao conjunto de teste

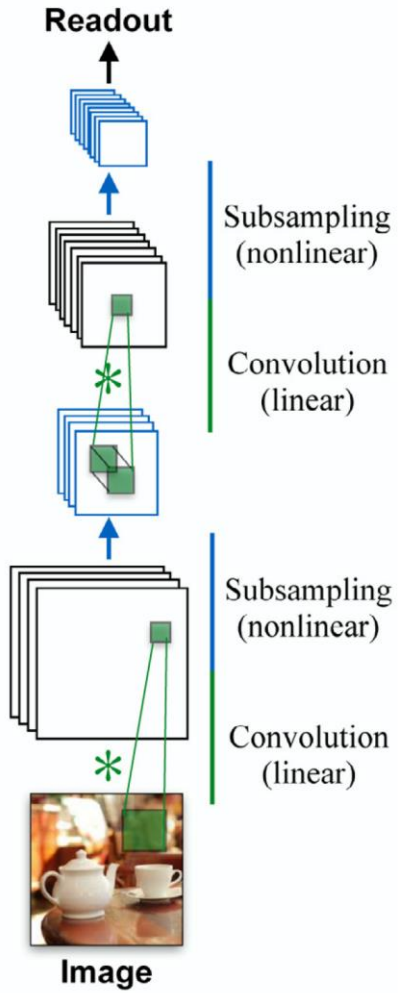


APRENDIZAGEM PROFUNDA

- A aprendizagem profunda (*deep learning*) é uma vertente de aprendizagem automática que utiliza redes neuronais artificiais com múltiplas camadas (*redes neuronais profundas*) para aprender padrões e relações complexas a partir de grandes volumes de dados
- É uma abordagem inspirada na estrutura e funcionamento do córtex cerebral, que possibilita a formação de representações dos dados em diferentes níveis de abstracção
- Tem tido um papel fundamental nos progressos recentes em diferentes áreas, nomeadamente:
  - Reconhecimento de imagem
  - Interfaces de fala
  - Sistemas de conversação
  - Veículos com condução autónoma



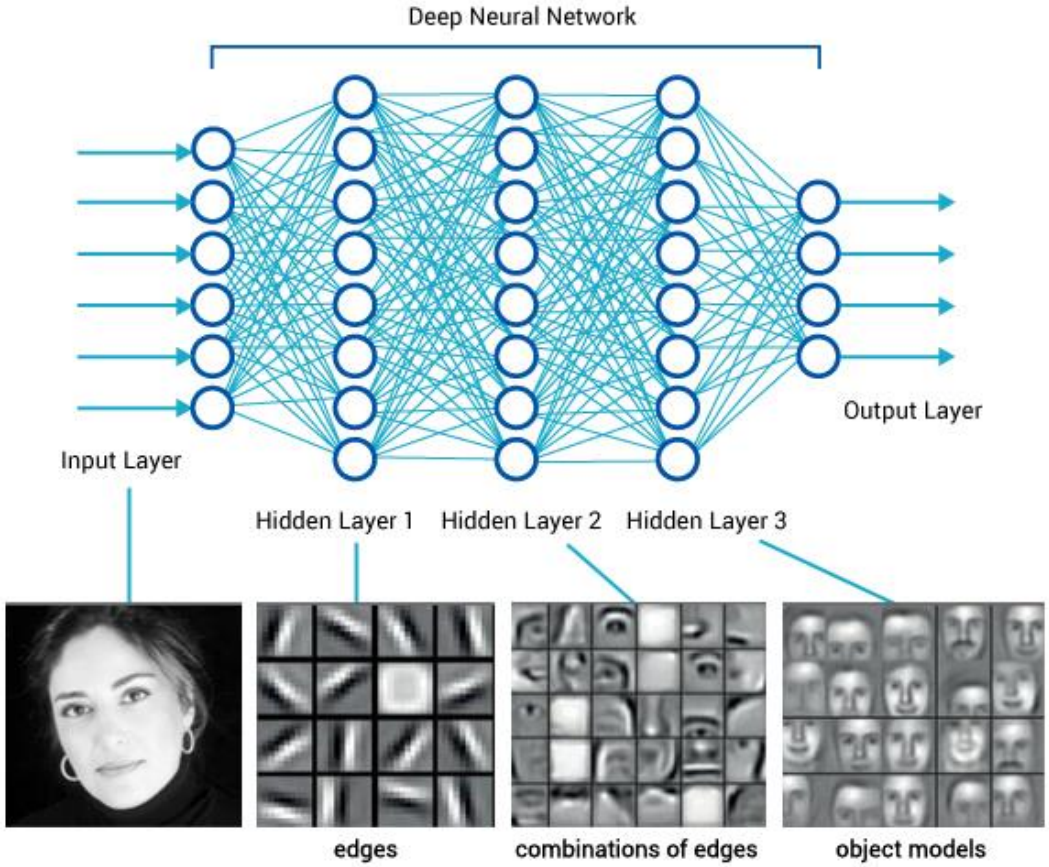
Exemplo de organização de regiões do córtex cerebral em colunas com diferentes níveis de neurónios



Exemplo de arquitectura de rede neuronal artificial com uma hierarquia de camadas com diferentes níveis de abstracção [Medathati *et al.*, 2016]

REDES NEURONAIS PROFUNDAS

As redes neuronais profundas (*deep neural networks*) são redes neuronais artificiais organizadas em múltiplas camadas heterogéneas, com diferentes tipos de neurónios e formas de organização das camadas e das respectivas ligações, o que possibilita a formação de representações dos dados em diferentes níveis de abstracção



REDES CONVOLUCIONAIS

As redes convolucionais são um tipo de rede neuronal vocacionada para processar dados caracterizados por padrões com invariantes espaciais, como é o caso de imagens, sendo por isso muito utilizadas em tarefas relacionadas com o processamento de imagem, por exemplo, detecção de objectos

As redes convolucionais são caracterizadas por três tipos principais de camadas:

- **Camadas Convolucionais**
  - Aplicam filtros (*kernels*) às entradas para extrair características
  - Cada filtro é responsável por detectar padrões específicos
- **Camadas de amostragem (*Pooling*)**
  - Reduzem a dimensionalidade dos dados, mantendo as características mais importantes
  - A amostragem pode ser realizada de diferentes formas, por exemplo, pelo valor máximo (*max pooling*) ou pelo valor médio (*average pooling*)
- **Camadas densas (completamente ligadas)**
  - Uma ou mais camadas finais completamente ligadas que produzem os valores de saída

REDES CONVOLUCIONAIS

As redes convolucionais têm sido fundamentais para o avanço em várias áreas da inteligência artificial, nomeadamente em tarefas relacionadas com visão computacional

Algumas vantagens das redes convolucionais:

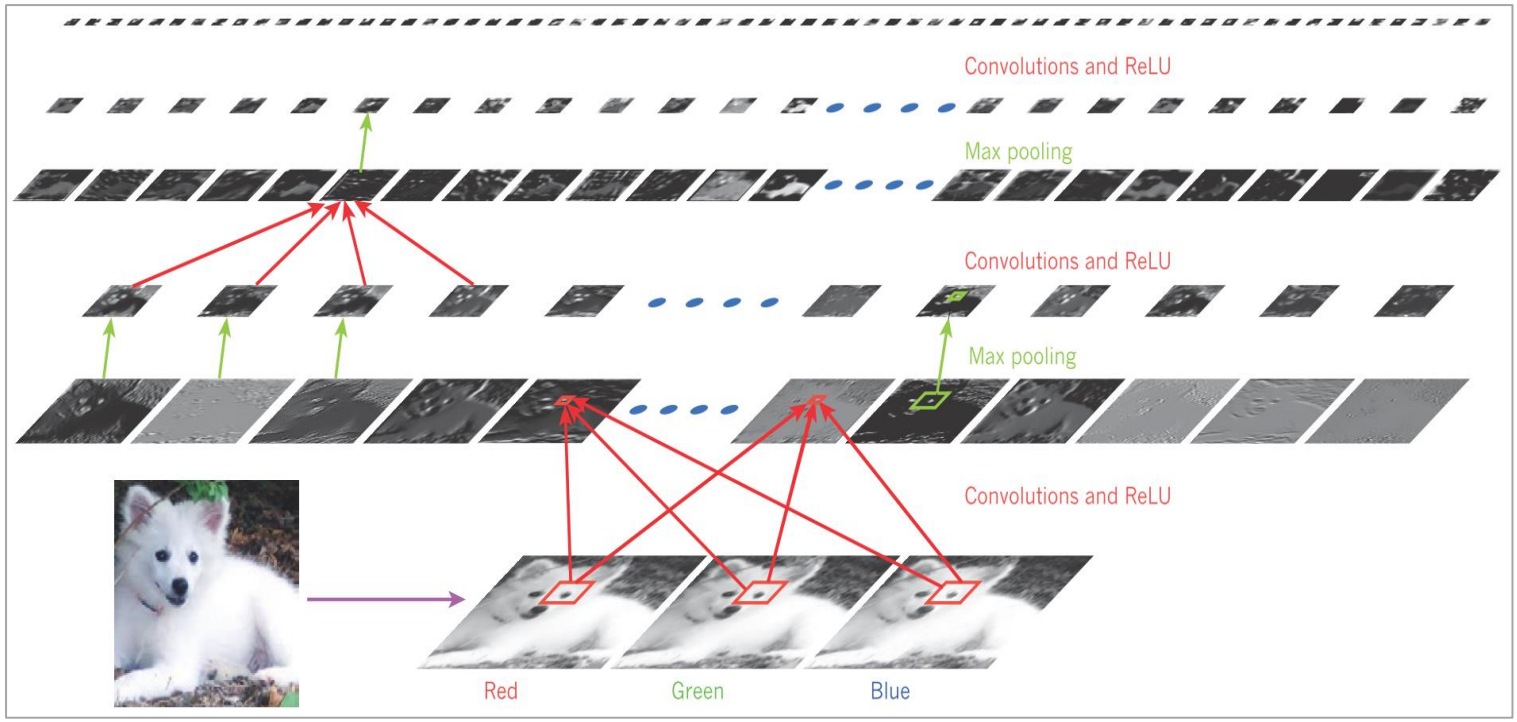
- **Invariância à Translação**
  - As redes convolucionais têm a capacidade de reconhecer padrões independentemente da sua posição nos dados, por exemplo, imagens
- **Redução de parâmetros**
  - A reutilização de filtros reduz o número total de parâmetros, tornando o treino mais eficiente
- **Extracção automática de características**
  - As redes convolucionais podem aprender automaticamente as características relevantes dos dados, eliminando a necessidade de definição manual de características

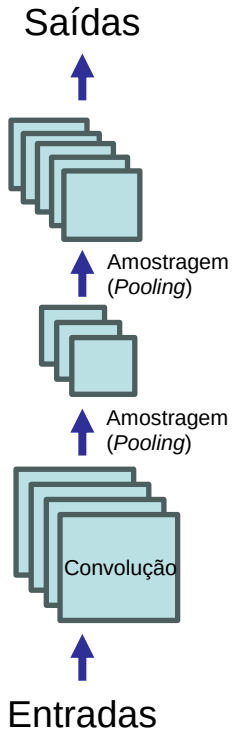
REDES CONVOLUCIONAIS

Exemplo de processamento numa rede convolucional:

Saídas de cada camada (horizontal) de uma rede convolucional aplicada a uma imagem e respectivas entradas RGB (vermelho, verde, azul), cada imagem rectangular é um mapa de características correspondente à saída para uma das características aprendidas, detectadas em cada uma das posições da imagem através de camadas convolucionais

A informação flui de baixo para cima, com as diferentes camadas actuando como detectores de padrões a diferentes níveis de abstracção



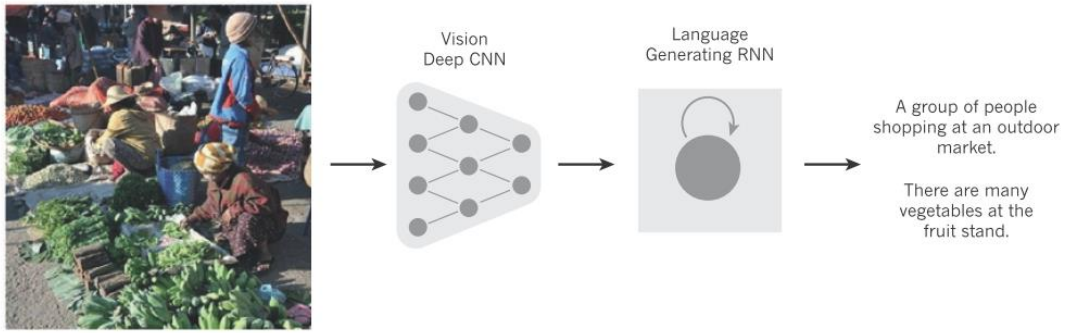


Adaptado de [LeCun *et al.*, 2015]

APRENDIZAGEM PROFUNDA

Exemplo:

Legendas geradas automaticamente por uma rede convolucional em conjunto com uma rede recorrente que traduz as representações de imagens em texto das legendas




A woman is throwing a **frisbee** in a park.


A **dog** is standing on a hardwood floor.



A **stop** sign is on a road with a mountain in the background


A little **girl** sitting on a bed with a teddy bear.


A group of **people** sitting on a boat in the water.


A giraffe standing in a forest with **trees** in the background.

[Vinyals *et al.*, 2014]

REDES NEURONAIS ARTIFICIAIS

- As *redes neuronais artificiais* são um modelo computacional derivado do estudo de sistemas nervosos biológicos, com **capacidade de aprendizagem**
- São sistemas computacionais **massivamente paralelos**, consistindo num número elevado de elementos de processamento (designados neurónios) interligados entre si de formas diversas
- **Características**:
  - Capacidade de aprendizagem
  - Capacidade de generalização
  - Adaptabilidade
  - Computação e representação distribuída
  - Paralelismo massivo
  - Robustez

NEURÓNIO - IMPLEMENTAÇÃO

Informação associada a um neurónio e respectivos valores iniciais com suporte para cálculo do momento

1.	iniciar(d, φ):	
2.	$d \in Z^+$	Dimensão de entrada (número ligações de entrada)
3.	$\varphi: R \rightarrow R$	Função de activação
4.	$\boldsymbol{w} = \text{random}(d, -1, 1)$	Vector de pesos das ligações de entrada
5.	$b = \text{random}(-1, 1)$	Pendor interno do neurónio
6.	$\Delta_{\boldsymbol{w}} = [0 \text{ for } d]$	**Vector de variação dos pesos**
7.	$\Delta_b = 0$	**Variação do pendor**
8.	$h = 0$	Soma da activação das entradas
9.	$y = 0$	Saída do neurónio
10.	$y' = 0$	Derivada da saída do neurónio

NEURÓNIO

Adaptar pesos das ligações de entrada e pendor interno com momento

1. adaptar$(\delta, \boldsymbol{y}^{n-1}, \alpha, \beta)$:
2. $\boldsymbol{M}_w = \beta \Delta_{\boldsymbol{w}}$ — Momento dos pesos
3. $\Delta_{\boldsymbol{w}} = -\alpha y' \delta \cdot \boldsymbol{y}^{n-1} + \boldsymbol{M}_w$ — Variação dos pesos
4. $\boldsymbol{w} = \boldsymbol{w} + \Delta_{\boldsymbol{w}}$ — Actualização dos pesos
5. $M_b = \beta \Delta_b$ — Momento do pendor
6. $\Delta_b = -\alpha y' \delta + M_b$ — Variação do pendor
7. $b = b + \Delta_b$ — Actualização do pendor

δ Componente de propagação do erro de saída do neurónio

$\boldsymbol{y}^{n-1}$ Vector de saída da camada anterior $n-1$

α Taxa de aprendizagem

β Factor de momento

CAMADA DENSA

Adaptar pesos e pendores dos neurónios da camada

1. adaptar$\left(\boldsymbol{\delta}^n, \boldsymbol{y}^{n-1}, \alpha, \beta\right)$:
2. for $j = 1$ to d_s:
3. $neuronios[j].adaptar\left(\delta_j^n, \boldsymbol{y}^{n-1}, \alpha, \beta\right)$

$\boldsymbol{\delta}^n$	Vector de erro de saída da camada n
δ_j^n	Componente j do vector de erro de saída da camada n
$\boldsymbol{y}^{n-1}$	Vector de saída da camada anterior $n-1$
α	Taxa de aprendizagem
β	Factor de momento

REDE NEURONAL

Retropropagar erro pelas camadas da rede, excepto a camada de entrada

1. retropropagar($\boldsymbol{\delta}^N, \alpha, \beta$):
2. $\quad \boldsymbol{\delta}^n = \boldsymbol{\delta}^N$
3. $\quad$ for $n = N$ to 2 step -1:
4. $\quad\quad \boldsymbol{y}^{n-1} = camadas[n-1].\boldsymbol{y}$
5. $\quad\quad d^{n-1} = camadas[n-1].d_s$
6. $\quad\quad d^n = camadas[n].d_s$
7. $\quad\quad neur^n = camadas[n].neuronios$
8. $\quad\quad \boldsymbol{\delta}^{n-1} = \left[\sum_{j=1}^{d^n} neur^n[j].w_i * \delta_j^n * neur^n[j].y' \text{ for } i = 1 \text{ to } d^{n-1} \right]$
9. $\quad\quad camadas[n].\text{adaptar}(\boldsymbol{\delta}^n, \boldsymbol{y}^{n-1}, \alpha, \beta)$
10. $\quad\quad \boldsymbol{\delta}^n = \boldsymbol{\delta}^{n-1}$

Camada de entrada apenas mantém entradas

Dimensão da camada *n* - 1

Dimensão da camada *n*

Neurónios da camada *n*

$\boldsymbol{\delta}^N$	Vector de variação de erro de saída da rede
$\boldsymbol{\delta}^n$	Vector de variação de erro de saída para cada camada *n*
δ_j	Componente *j* do vector de variação de erro de saída para cada camada
α	Taxa de aprendizagem
β	Factor de momento

REDE NEURONAL

Adaptar parâmetros da rede

1. adaptar($\boldsymbol{x}, \boldsymbol{y}, \alpha, \beta$):
2. $\quad \boldsymbol{y}^N = \text{propagar}(\boldsymbol{x})$
3. $\quad \boldsymbol{\delta}^N = \text{delta_saída}(\boldsymbol{y}^N, \boldsymbol{y})$
4. $\quad \text{retropropagar}(\boldsymbol{\delta}^N, \alpha, \beta)$
5. $\quad K = \left|\boldsymbol{\delta}^N\right|$
6. $\quad \varepsilon = \frac{1}{K}\sum_{k=1}^{K}\left(\delta_k^N\right)^2$
7. $\quad \text{return } \varepsilon$

$\boldsymbol{x}$	Vector de treino de entrada
$\boldsymbol{y}$	Vector de treino de saída
$\boldsymbol{y}^N$	Vector de saída gerado pela rede
$\boldsymbol{\delta}^N$	Vector de variação de erro de saída
ε	Erro médio de saída da rede
K	Dimensão do vector de perda
α	Taxa de aprendizagem
β	Factor de momento

REDE NEURONAL

Treinar a rede neuronal

```
1.  treinar(X, Y, n_épocas, ε_max, α, β):
2.      for n_épocas:
3.          ε = 0                                  ε - erro de treino por época
4.          for x ∈ X, y ∈ Y:
5.              ε_x = adaptar(x, y, α, β)          ε_x - erro de treino por entrada x
6.              ε = max(ε, ε_x):
7.          if ε ≤ ε_max:
8.              break
```

$\boldsymbol{X}$	Conjunto de entradas $\boldsymbol{x}$
$\boldsymbol{Y}$	Conjunto de saídas $\boldsymbol{y}$
$n_{épocas}$	Número de épocas de treino
ε_{max}	Erro máximo
α	Taxa de aprendizagem
β	Factor de momento

BIBLIOGRAFIA

[Aggarwal, 2018]
C. Aggarwal, Neural Networks and Deep Learning
Springer, 2018

[Munakata, 1998]
T. Munakata, *Fundamentals of the New Artificial Intelligence*, Springer, 1998

[Winston, 1992]
P. Winston, *Artificial Intelligence*, 3rd Edition, Addison-Wesley, 1992

[Raizer *et al.*, 2009]
K. Raizer, H. Idagawa, E. Nobrega, L. Ferreira, *Training and Applying a Feedforward Multilayer Neural Network in GPU*, CILAMCE, 2009

[Gutierrez-Osuna, 2005]
R. Gutierrez-Osuna, *Introduction to Pattern Analysis*, Texas A&M University, 2005

[Vinyals *et al.*, 2014]
O. Vinyals, A. Toshev, S. Bengio, D. Erhan, *Show and tell: a neural image caption generator*, In Proceedings of the International Conference on Machine Learning, 2014

[Medathati *et al.*, 2016]
N. Medathati, H. Neumann, G. Masson, P. Kornprobst. (2016). *Bio-Inspired Computer Vision: Towards a Synergistic Approach of Artificial and Biological Vision,* Computer Vision and Image Understanding, 2016

[LeCun *et al*., 2015]
Y. LeCun, Y. Bengio, G. Hinton, *Deep Learning*, Nature 521, 2015

[Berner *et al*., 2023]
J. Berner, P. Grohs, G. Kutyniok, P. Petersen, *The Modern Mathematics of Deep Learning*, Mathematical Aspects of Deep Learning, Cambridge University Press, 2023